

CodeFlow AI

Autonomous Multi-Agent Code Review & Refactoring Engine

Technical Architecture, Working Pipeline & System Specification Manual
Version 1.0.0 | Stack: LangGraph, Groq LLaMA 3.3 70B, FastAPI, React Vite

1. Executive Summary & Core System Objectives

CodeFlow AI is an enterprise-grade autonomous multi-agent system designed to replace slow manual code reviews with instant, automated, self-healing code quality enforcement. Driven by **LangGraph** state machines and ultra-low-latency **Groq LLMs** (LLaMA 3.3 70B / Mixtral), CodeFlow AI continuously reviews code, detects security vulnerabilities and logical bugs, generates targeted patches, and executes isolated unit tests until full convergence is achieved.

Feature / Module	Technical Mechanism	Impact / Business Benefit
Autonomous Reviewer	Parses code into structured JSON assessing bugs, security (SQLi/Path Traversal), performance & style.	Instant objective code quality scoring (1-10) with exact line number defect mapping.
Self-Healing Developer	Consumes review issues, synthesizes clean patches, and executes sandboxed Pytest runs.	Fully automated refactoring without manual developer effort.
Stateful Graph Control	LangGraph cyclic workflow orchestrates multi-agent iterations & state history persistence.	Guarantees bounded loops with max-iteration guards and clean termination.
Real-Time SSE Telemetry	FastAPI Server-Sent Events stream live progress, defect tables, and test results.	Zero-latency streaming UI with glassmorphic React dashboard.

2. System Architecture & Visual Flow

The CodeFlow AI system is structured cleanly into three operational tiers: the Presentation Tier (React Dashboard), the Application API Tier (FastAPI with SSE streaming), and the Autonomous Agentic Orchestration Tier (LangGraph + Sandboxed Workspace).

TIER 1: PRESENTATION LAYER	TIER 2: API & STREAMING LAYER	TIER 3: AGENTIC CORE LAYER
React + Vite Dashboard - Glassmorphism Design - Server-Sent Events Listener - Side-by-Side Code Diff Viewer - Real-Time Quality Metric Stream	FastAPI Server (`api.py`) - POST `/review` (Text input) - POST `/review/upload` (Files) - GET `/health` System Monitor - StreamingResponse Generator	LangGraph Orchestrator Reviewer Agent Node: Code audit Developer Agent Node: Patching Pytest Tooling: Unit testing Workspace Sandbox: File I/O

3. Detailed Step-by-Step Working Pipeline

The end-to-end execution lifecycle consists of 6 sequential phases, operating in a closed loop until code quality standards are met or cycle limits expire.

Phase 1: Ingestion & Workspace Isolation

- Payload Receipt:** The user submits raw code string or uploads a file via `/review` or `/review/upload`.
- Filename Normalization:** The API assigns a filename (`test\_code\_sample.py` if test signatures exist, else `code\_sample.py`).
- Disk Persistence:** File is written to `workspace/{filename}` via `write\_file`, enforcing strict path sandboxing.

Phase 2: State Graph Initialization (`CodeFlowState`)

The system initializes a thread-safe `CodeFlowState` dictionary holding snapshots and history:

```
state: CodeFlowState = {
  "original_code": code,
  "current_code": code,
  "filename": filename,
  "review_history": [],
  "patch_history": [],
  "iteration": 0,
  "max_iterations": max_iterations, # Default: 3
  "final_status": None,
  "error": None
}
```

Phase 3: Reviewer Agent Node Execution (`reviewer\_node`)

Execution enters `reviewer\_node`. The Reviewer Agent leverages `ChatGroq(model\_name='openai/gpt-oss-20b', temperature=0.1)`:

- **System Prompt Enforcement:** Evaluates code for bugs, security vulnerabilities (SQL injection, path traversal, hardcoded secrets), performance traps, and readability.
- **JSON Deserialization & Robust Auto-Repair:**
 - a. Strips markdown fences (````json ````).
 - b. Extracts text between first `{` and last `}`.
 - c. **Truncation Auto-Repair:** Calculates open vs closed braces/brackets and appends missing `}` / `]` to fix cut-off LLM outputs.
 - d. **Regex Fallback:** If JSON parsing fails, regex scans text for severity keywords (`CRITICAL`, `HIGH`, `MEDIUM`) to build structured `Issue` records.
 - e. Instantiates `ReviewResult` and appends it to `review\_history`.
- **State Update:** Increments `iteration += 1`. If `decision == 'APPROVED'`, sets `final\_status = 'APPROVED'`. Yields SSE `type: reviewer` event.

Phase 4: Conditional Workflow Routing (`should\_continue`)

LangGraph inspects current state to decide graph progression:

```
def should_continue(state: CodeFlowState) -> str:
    if state.get("final_status") == "APPROVED":
        return END # Code is fully approved by Reviewer!

    iteration = state.get("iteration", 0)
    max_iterations = state.get("max_iterations", 3)

    if iteration >= max_iterations:
        state["final_status"] = "MAX_ITERATIONS_REACHED"
        return END # Loop threshold reached

    return "developer" # Route to Developer Agent for patching
```

Phase 5: Developer Agent Node Execution (`developer\_node`)

When routed to `developer\_node`, the Developer Agent performs refactoring:

- **Issue Analysis:** Loads issue list & summary from `review\_history[-1]` and current code from `workspace`.
- **LLM Patch Synthesis:** Invokes `ChatGroq(temperature=0.2)` requesting pure corrected code with zero markdown wrapping.
- **File Modification:** Writes patched code to disk via `write\_file(filename, fixed\_code)`.
- **Automated Pytest Execution:** Calls `run\_tests(filename)` executing `pytest -v` with a 30s timeout.
- **Metrics Extraction:** Uses regex (`(\\d+) passed`, `(\\d+) failed`) to capture pass/fail counts into `TestMetrics`.
- **History & State Update:** Appends `PatchResult` to `patch\_history`, updates `current\_code`, and yields SSE `type: developer` event.

Phase 6: Loop Recirculation & Termination Event

An edge connects `developer\_node` back to `reviewer\_node`. The Reviewer Agent evaluates the newly patched code. When the graph reaches `END`, FastAPI yields a final SSE event `{type: 'done', final\_status: ..., current\_code: ...}` and closes the stream cleanly.

4. Data Schemas & State Data Models

CodeFlow AI enforces strong Pydantic schemas to ensure data integrity across agents and SSE outputs:

Schema Name	Fields & Data Types	Role & Constraints
CodeFlowState (TypedDict)	original_code: str current_code: str filename: str review_history: List[ReviewResult] patch_history: List[PatchResult] iteration: int max_iterations: int final_status: Optional[str]	Central state object maintained by LangGraph throughout execution cycle.
Issue (Pydantic)	severity: CRITICAL HIGH MEDIUM LOW INFO category: BUG SECURITY PERFORMANCE STYLE... line: Optional[int] title: str description: str suggestion: Optional[str] must_fix: bool	Structured breakdown of individual code defects found by Reviewer.
ReviewResult (Pydantic)	decision: APPROVED REJECTED issues: List[Issue] summary: str score: int (1 to 10)	Output of a review pass including numerical quality rating.
PatchResult (Pydantic)	status: PATCHED FAILED ERROR changes: List[str] tests: TestMetrics (passed: int, failed: int)	Summary of code edits made by Developer and test results.

5. Agent Tool Matrix & Security Sandboxing

To prevent arbitrary file access or malicious system calls, all agent file operations are routed through strict path resolution guards in ``tools/file_tools.py``.

```
WORKSPACE_DIR = os.path.join(os.getcwd(), "workspace")

def get_safe_path(filename: str) -> str:
    _ensure_workspace()
    safe_path = os.path.abspath(os.path.join(WORKSPACE_DIR, filename))
    if not safe_path.startswith(WORKSPACE_DIR):
        raise ValueError(f"Access denied. Path {filename} is outside the workspace.")
    return safe_path
```

Tool Name	Signature	Security Guard & Behavior
read_workspace_file	read_file(filename: str) -> str	Validates path via <code>`get_safe_path`</code> . Reads workspace file.
write_workspace_file	write_file(filename, content)	Validates path. Writes content safely to <code>`workspace/`</code> .
run_workspace_tests	run_tests(target: str = None)	Executes <code>`pytest -v`</code> in subprocess with 30s timeout.
search_workspace_file	list_files() matching query	Scans relative files strictly inside <code>`workspace/`</code> .

6. API Endpoints & SSE Event Specifications

FastAPI streams real-time updates using Server-Sent Events (``text/event-stream``):

SSE Event Type	Triggering Node	Sample Payload Schema
<code>`type: reviewer`</code>	Emitted after <code>`reviewer_node`</code> completes evaluation.	<code>{ "type": "reviewer", "iteration": 1, "score": 6, "decision": "REJECTED", "summary": "SQL injection flaw on line 12", "issues": [{ "severity": "CRITICAL", ... }] }</code>
<code>`type: developer`</code>	Emitted after <code>`developer_node`</code> completes patch & tests.	<code>{ "type": "developer", "iteration": 1, "status": "PATCHED", "changes": [...], "tests": { "passed": 3, "failed": 0 }, "current_code": "def safe_query(...)..." }</code>
<code>`type: done`</code>	Emitted when graph reaches <code>`END`</code> state.	<code>{ "type": "done", "final_status": "APPROVED", "current_code": "..." }</code>
<code>`type: error`</code>	Emitted if an exception escapes workflow execution.	<code>{ "type": "error", "message": "...", "detail": "Traceback string..." }</code>

7. Deployment, Installation & Quality Assurance

CodeFlow AI includes a full Pytest test suite (31+ tests) covering schemas, tools, nodes, state graphs, and API endpoints, along with a live E2E runner (``run_live_test.py``).

```
# 1. Environment Setup
echo "GROQ_API_KEY=your_groq_api_key_here" > .env

# 2. Install Python Dependencies
pip install -r requirements.txt

# 3. Launch FastAPI Backend Server (Runs on http://127.0.0.1:8000)
python -m uvicorn api:app --host 127.0.0.1 --port 8000 --reload

# 4. Launch React Frontend Dashboard (Runs on http://localhost:5173)
cd frontend && npm install && npm run dev

# 5. Run Automated Pytest Suite & E2E Pipeline Verification
pytest tests/ -v
python run_live_test.py
```

■ Enterprise CI/CD Integration Note:

Integrate CodeFlow AI into GitHub Actions or GitLab CI by making an SSE call to `http://127.0.0.1:8000/review/upload` during pull request builds. Set `max_iterations=3` to enforce bounded execution times while generating automated review artifacts.